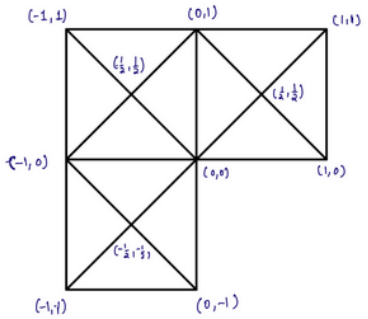
1. Consider the following triangulation and write a MATLAB code for



(a) Create the Coordinate vector.

(b) Create the Elements matrix.

(c) Give a list boundary edges.

(d) Plot the triangulation.

(e) Create a vector of centroid of each triangle/element.

(f) Create a vector of area of each triangle/element.

(g) Find $\int_0^1 \int_0^1 e^{x+y} dxdy$ using the triangle mid-point rule.

(h) Print the list of all edges.

(i) Print the list of all boundary nodes.

(j) Print the list of all interior nodes.

(k) Print the list of mid-points of all the edges.

(l) Find a list of all triangles sharing node $(0, 0)$. (Hint- use the "find" command from MATLAB)

(m) Find the triangles that share the edge joined by (0,1)-(0,0).

Write the code such that it will work for any triangulations.

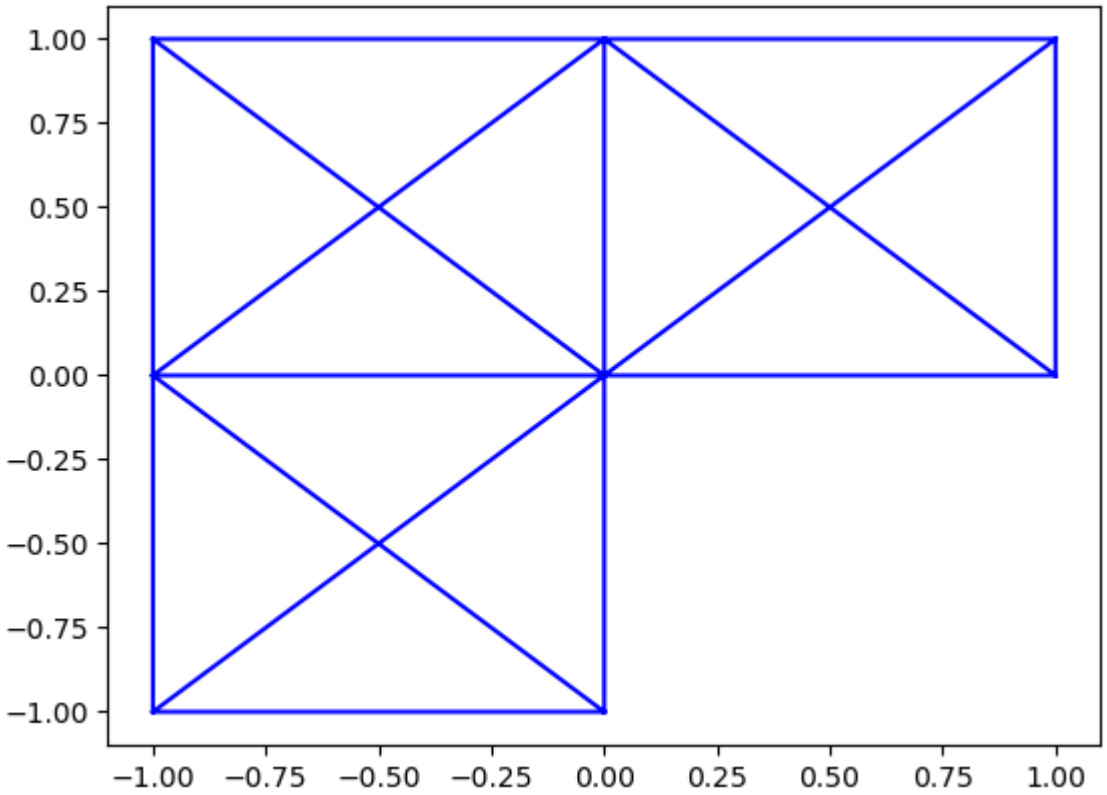
In [1]:
```
using PyPlot, LinearAlgebra
```

In [2]:
```
# for L-shaped domain
# PART a, b, c, d
coord = [-1 -1; 0 -1;
            0 0; 1 0;
            1 1; 0 1;
          -1 1; -1 0;
           -0.5 -0.5;
            -0.5 0.5;
             0.5 0.5]

Elem = [1 2 9; 2 3 9;
        3 8 9; 8 1 9;
        8 3 10; 3 6 10;
        6 7 10; 7 8 10;
        3 4 11; 4 5 11;
        5 6 11; 6 3 11]

DB = [1 2; 2 3;
      3 4; 4 5;
      5 6; 6 7;
      7 8; 8 1]

# since triplot takes 0-based indexing
triangles = Elem .- 1
plt.triplot(coord[:, 1], coord[:, 2], triangles, color="blue")
```



Out[2]:
```
2-element Vector{PyCall.PyObject}:
 PyObject <matplotlib.lines.Line2D object at 0x7fe2adb64230>
 PyObject <matplotlib.lines.Line2D object at 0x7fe2a41104d0>
```

In [3]:
```
# PART e, f, g
function f(cen)
    return exp(cen[1] + cen[2])
end

area_total = 0.0
for i in 1:size(Elem, 1)

    # area of the triangle
    coordTi = coord[Elem[i, :], :]
    det_ = [1 1 1; transpose(coordTi)]
    area_elem = 0.5*det(det_)
    a1 = sum(coordTi[:, 1])
    b1 = sum(coordTi[:, 2])

    # Centroid of each triangle
    centroid = [a1/3, b1/3]
    println("Centroid of element $i is $centroid")
    global area_total += area_elem*f(centroid)
end

println("\nTotal area using mid-point rule: $area_total")
println("Exact area: 4.439\n")
```

```
Centroid of element 1 is [-0.5, -0.8333333333333334]
Centroid of element 2 is [-0.16666666666666666, -0.5]
Centroid of element 3 is [-0.5, -0.16666666666666666]
Centroid of element 4 is [-0.8333333333333334, -0.5]
Centroid of element 5 is [-0.5, 0.16666666666666666]
Centroid of element 6 is [-0.16666666666666666, 0.5]
Centroid of element 7 is [-0.5, 0.8333333333333334]
Centroid of element 8 is [-0.8333333333333334, 0.5]
Centroid of element 9 is [0.5, 0.16666666666666666]
Centroid of element 10 is [0.8333333333333334, 0.5]
Centroid of element 11 is [0.5, 0.8333333333333334]
Centroid of element 12 is [0.16666666666666666, 0.5]

Total area using mid-point rule: 4.315279964273026
Exact area: 4.439
```

In [4]:

```
# PART h, i, j, k

println("List of all edges:")
edges = []
for i in 1:size(Elem, 1)
    n1, n2, n3 = Elem[i, 1], Elem[i, 2], Elem[i, 3]
    println("[$n1 $n2; $n2 $n3; $n3 $n1]")
    push!(edges, (min(n1, n2), max(n1, n2)))
    push!(edges, (min(n2, n3), max(n2, n3)))
    push!(edges, (min(n3, n1), max(n3, n1)))
end

println()
edges = unique!(edges)
# println(edges)
for i in 1:length(edges)
    p1, p2 = edges[i][1], edges[i][2]
    _p1 = coord[p1, :]
    _p2 = coord[p2, :]
    midpoint = (_p1 + _p2) ./ 2
    println("Midpoint of edge [$p1, $p2] is $midpoint.")
end

println("\nList of all boundary nodes: ", unique(DB))
println("List of all interior nodes: ", setdiff(1:size(coord, 1), unique(D
```

```
List of all edges:
[1 2; 2 9; 9 1]
[2 3; 3 9; 9 2]
[3 8; 8 9; 9 3]
[8 1; 1 9; 9 8]
[8 3; 3 10; 10 8]
[3 6; 6 10; 10 3]
[6 7; 7 10; 10 6]
[7 8; 8 10; 10 7]
[3 4; 4 11; 11 3]
[4 5; 5 11; 11 4]
[5 6; 6 11; 11 5]
[6 3; 3 11; 11 6]

Midpoint of edge [1, 2] is [-0.5, -1.0].
Midpoint of edge [2, 9] is [-0.25, -0.75].
Midpoint of edge [1, 9] is [-0.75, -0.75].
Midpoint of edge [2, 3] is [0.0, -0.5].
Midpoint of edge [3, 9] is [-0.25, -0.25].
Midpoint of edge [3, 8] is [-0.5, 0.0].
Midpoint of edge [8, 9] is [-0.75, -0.25].
Midpoint of edge [1, 8] is [-1.0, -0.5].
Midpoint of edge [3, 10] is [-0.25, 0.25].
Midpoint of edge [8, 10] is [-0.75, 0.25].
Midpoint of edge [3, 6] is [0.0, 0.5].
Midpoint of edge [6, 10] is [-0.25, 0.75].
Midpoint of edge [6, 7] is [-0.5, 1.0].
Midpoint of edge [7, 10] is [-0.75, 0.75].
Midpoint of edge [7, 8] is [-1.0, 0.5].
Midpoint of edge [3, 4] is [0.5, 0.0].
Midpoint of edge [4, 11] is [0.75, 0.25].
Midpoint of edge [3, 11] is [0.25, 0.25].
Midpoint of edge [4, 5] is [1.0, 0.5].
Midpoint of edge [5, 11] is [0.75, 0.75].
Midpoint of edge [5, 6] is [0.5, 1.0].
Midpoint of edge [6, 11] is [0.25, 0.75].

List of all boundary nodes: [1, 2, 3, 4, 5, 6, 7, 8]
List of all interior nodes: [9, 10, 11]
```

In [5]:

```julia
# PART 1
# search for node (0,0)
search_node = findfirst(x -> x == [0, 0], eachrow(coord))
trig_with_node = []
println("List of triangles sharing the node (0.0, 0.0):\n")
for i in 1:size(Elem, 1)
    if search_node == Elem[i, 1] || search_node == Elem[i, 2] || search_no
        println(Elem[i, :])
    end
end
```

```
List of triangles sharing the node (0.0, 0.0):

[2, 3, 9]
[3, 8, 9]
[8, 3, 10]
[3, 6, 10]
[3, 4, 11]
[6, 3, 11]
```

In [6]:
```
# PART m
node1 = findfirst(x -> x == [0, 1], eachrow(coord))
node2 = findfirst(x -> x == [0, 0], eachrow(coord))

println("List of triangles sharing the edge joined by (0, 1) - (0, 0):\n"
for i in 1:size(Elem, 1)
    if (node1 in Elem[i, :]) && (node2 in Elem[i, :])
        println(Elem[i, :])
    end
end
```

```
List of triangles sharing the edge joined by (0, 1) - (0, 0):

[3, 6, 10]
[6, 3, 11]
```

Question 2

2. Plotting the function using "trisurf" command

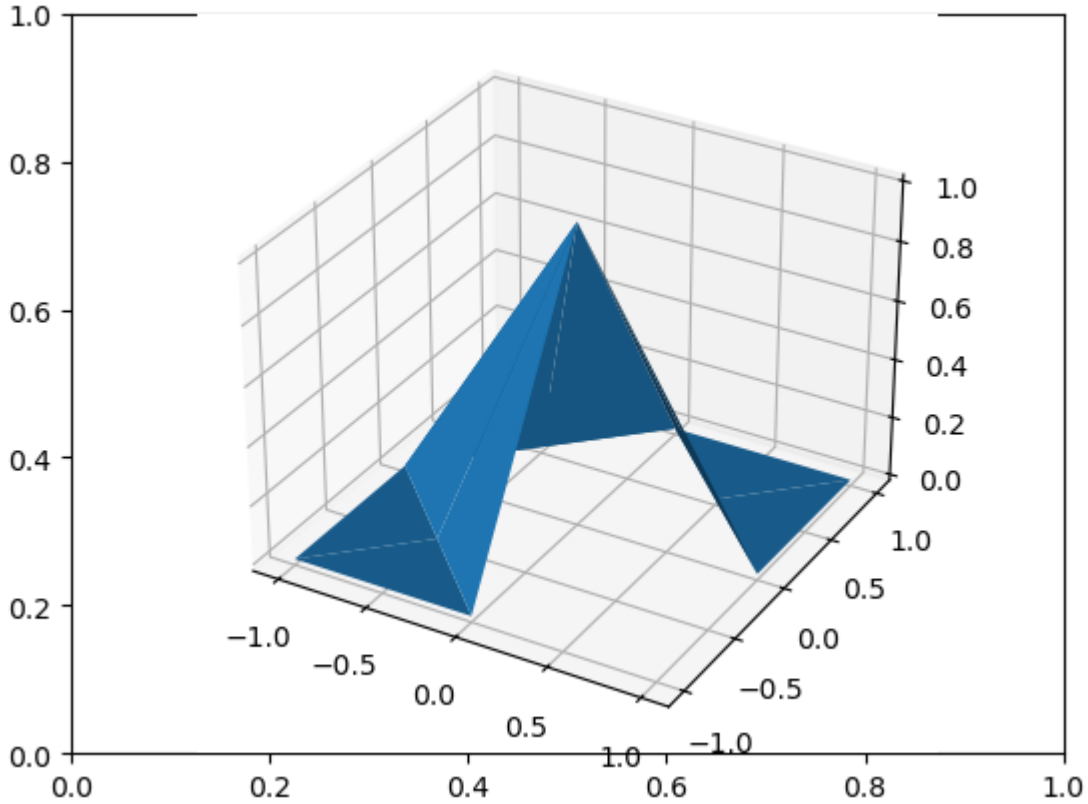
   (a) Plot a piecewise linear function, which takes value one at $(0, 0)$ and zero at other node points.

   (b) Plot a piecewise linear function, which takes value one at $(0.5, 0.5)$ and zero at other node points.

   (c) Initialize a square matrix $A \in \mathbb{R}^{N \times N}$ with zero entries, where N denotes the number of nodes. Define a matrix $B \in \mathbb{R}^{3 \times 3}$ whose entries are all equal to one. For each triangle in the mesh, assemble the local matrix B into the global matrix A using the corresponding global node indices, and repeat this process by looping over all triangles.

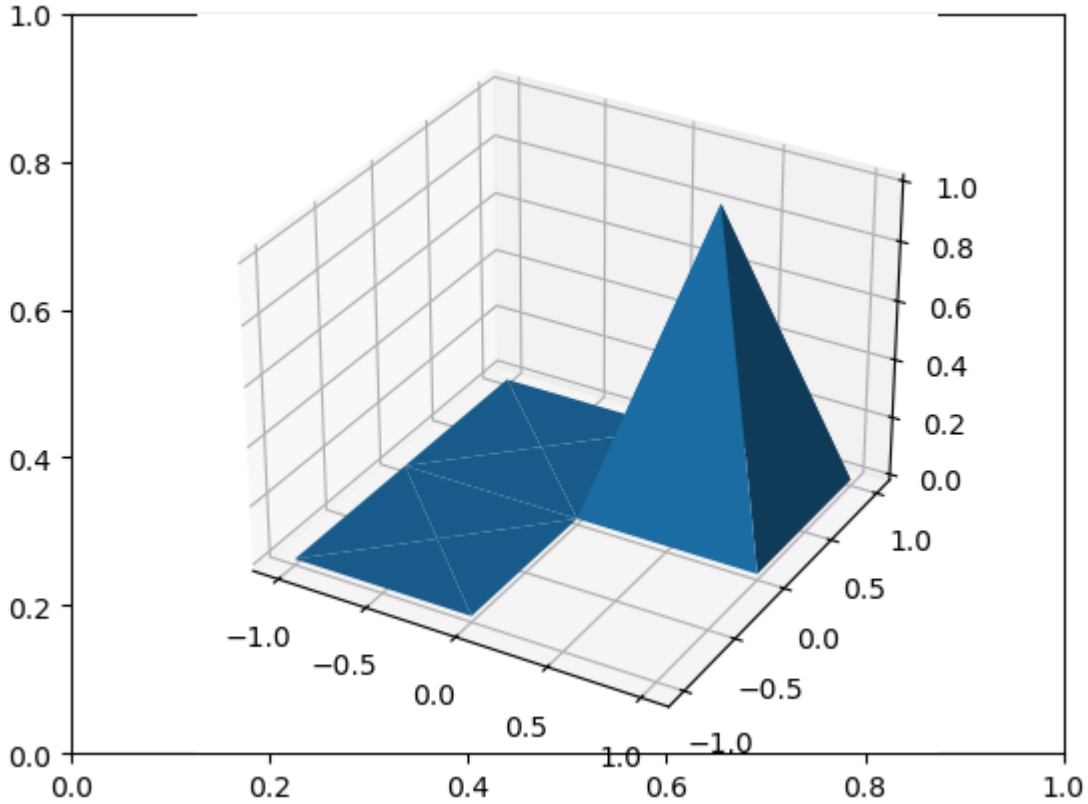
   (d) Construct a matrix whose first column corresponds to the node indices, and whose second column contains the number of triangles associated with each node.

In [7]:
```
# PART a
function f1(x)
    if x == [0, 0]
        return 1.0
    else
        return 0.0
    end
end
func_value = [f1(i) for i in eachrow(coord)]
plot_trisurf(coord[:, 1], coord[:, 2], triangles, func_value)
```



Out[7]: PyObject <mpl_toolkits.mplot3d.art3d.Poly3DCollection object at 0x7fe29ff21310>

In [8]:
```
# PART b
function f2(x)
    if x == [0.5, 0.5]
        return 1.0
    else
        return 0.0
    end
end
func_value = [f2(i) for i in eachrow(coord)]
plot_trisurf(coord[:, 1], coord[:, 2], triangles, func_value)
```



Out[8]: PyObject <mpl_toolkits.mplot3d.art3d.Poly3DCollection object at 0x7fe2a001c530>

In [9]:
```
# PART c
N = size(coord, 1)
A = zeros(N, N)
B = ones(3, 3)

for i in 1:size(Elem, 1)
    A[Elem[i, :], Elem[i, :]] += B
end
A
```

Out[9]:
```
11×11 Matrix{Float64}:
 2.0  1.0  0.0  0.0  0.0  0.0  0.0  1.0  2.0  0.0  0.0
 1.0  2.0  1.0  0.0  0.0  0.0  0.0  0.0  2.0  0.0  0.0
 0.0  1.0  6.0  1.0  0.0  2.0  0.0  2.0  2.0  2.0  2.0
 0.0  0.0  1.0  2.0  1.0  0.0  0.0  0.0  0.0  0.0  2.0
 0.0  0.0  0.0  1.0  2.0  1.0  0.0  0.0  0.0  0.0  2.0
 0.0  0.0  2.0  0.0  1.0  4.0  1.0  0.0  0.0  2.0  2.0
 0.0  0.0  0.0  0.0  0.0  1.0  2.0  1.0  0.0  2.0  0.0
 1.0  0.0  2.0  0.0  0.0  0.0  1.0  4.0  2.0  2.0  0.0
 2.0  2.0  2.0  0.0  0.0  0.0  0.0  2.0  4.0  0.0  0.0
 0.0  0.0  2.0  0.0  0.0  2.0  2.0  2.0  0.0  4.0  0.0
 0.0  0.0  2.0  2.0  2.0  2.0  0.0  0.0  0.0  0.0  4.0
```

In [10]:
```
N = size(coord,1)
count_triangles = zeros(Int, N)

for i in 1:N # loop over nodes
    for j in 1:size(Elem, 1) # loop over elements
        if i in Elem[j, :]
            count_triangles[i] += 1
        end
    end
```

```julia
end

nodes_triangles = hcat(1:N, count_triangles)
println("Node index    Number of associated triangles")

for i in 1:size(nodes_triangles,1)
    println(nodes_triangles[i,1], "\t\t\t", nodes_triangles[i,2])
end
```

```
Node index    Number of associated triangles
1                       2
2                       2
3                       6
4                       2
5                       2
6                       4
7                       2
8                       4
9                       4
10                      4
11                      4
```